# Atividade Extraclasse: Fundamentos de TypeScript & POO

Lista Prática de Exercícios (10 Questões Diretas) — Tipagem, Encapsulamento, Herança e Interfaces

**Disciplina:** Programação Orientada a Objetos **Linguagem:** TypeScript

**Nome do Aluno:** _____ **Data:** _____

### Exercício 1: Tipos Básicos e Modificadores de Acesso

Fundamentos & Encapsulamento

Crie uma classe simples chamada `Pessoa` com a seguinte estrutura:

- Atributo público: `nome` (string).
- Atributo privado: `_idade` (number).
- Atributo protegido: `_cpf` (string).
- Crie um método `getIdade(): number` que retorna a idade e um método `setIdade(novaIdade: number): void` que só aceita números positivos.

### Exercício 2: Métodos Getters e Setters (Properties)

Encapsulamento

Implemente uma classe `Produto` utilizando os acessores do TypeScript:

- Atributos privados: `_nome` (string) e `_preco` (number).
- Utilize o acessor `get preco()` para retornar o preço.
- Utilize o acessor `set preco(valor: number)` que impede que o preço seja definido como um valor menor que zero.

### Exercício 3: Herança Simples e Sobrescrita de Método

Herança

Crie uma hierarquia simples de animais:

- Classe base `Animal` com o atributo `nome` (string) e o método `emitirSom(): string` que retorna `"Som genérico"`.
- Classe `Cachorro` que herda de `Animal` e sobrescreve o método `emitirSom()` para retornar `"Au Au!"`.
- Classe `Gato` que herda de `Animal` e sobrescreve para retornar `"Miau!"`.

### Exercício 4: Union Types nas Propriedades

Type Unions

Crie uma classe `Identificador` que lida com diferentes formatos de ID:

- Atributo privado `_codigo` do tipo `string | number` (Union Type).
- Construtor que aceita tanto `string` quanto `number`.
- Método `exibirCodigo(): void` que imprime o código na tela. Se for string, exiba em letras maiúsculas.

### Exercício 5: Criação e Implementação de Interface

Interfaces

Modele um contrato básico de autenticação:

- Crie uma interface `Autenticavel` com o método `autenticar(senha: string): boolean`.
- Crie a classe `Usuario` que implementa a interface `Autenticavel`.
- A classe deve ter o atributo privado `senhaCorreta` e validar a senha digitada no método `autenticar`.

### Exercício 6: Simulação de Múltipla Herança com Interfaces

Interfaces & Dobre Herança

Implemente múltiplos contratos em uma mesma classe:

- Crie a interface `Imprimivel` com o método `imprimir(): void`.
- Crie a interface `Exportavel` com o método `exportarJSON(): string`.
- Crie a classe `Relatorio` que implementa **ambas** as interfaces, fornecendo o código para ambos os métodos.

### Exercício 7: Classes Abstratas

Abstração & Herança

Trabalhe com o conceito de classes abstratas:

- Crie a classe abstrata `FiguraGeometrica` com o método abstrato `calcularArea(): number`.
- Crie a classe `Quadrado` (com o atributo `lado`) herdando de `FiguraGeometrica` e implementando o cálculo da área.
- Crie a classe `Circulo` (com o atributo `raio`) também herdando de `FiguraGeometrica` e implementando a área.

### Exercício 8: Type Aliases e Union Types Literal

TypeScript Fundamental

Crie tipos customizados com Union Types para gerenciar o status de um pedido:

- Crie o Type Alias `StatusPedido = "pendente" | "pago" | "enviado" | "cancelado"`.
- Crie a classe `Pedido` que possui um atributo privado `status` do tipo `StatusPedido`.
- Crie o método `atualizarStatus(novoStatus: StatusPedido): void` para alterar o status do pedido.

### Exercício 9: Arrays com Tipos e Polimorfismo

Polimorfismo

Crie um gerenciador que utiliza polimorfismo com base na interface do Exercício 5:

- Crie a interface `Notificavel` com o método `enviarMensagem(texto: string): void`.
- Crie as classes `NotificacaoEmail` e `NotificacaoSMS` implementando `Notificavel`.
- Crie uma função ou classe `GerenciadorNotificacoes` que recebe um array de `Notificavel[]` e dispara mensagens para todos simultaneamente.

### Exercício 10: Generics Básicos com Classes

Generics & TS Avançado

Crie uma estrutura flexível e reativa para armazenar dados de qualquer tipo:

- Crie a classe genérica `Caixa<T>`.
- Adicione um atributo privado `_conteudo` do tipo `T`.
- Implemente os métodos `guardar(item: T): void` e `retirar(): T`.
- Teste a classe instanciando uma `Caixa<string>` para textos e uma `Caixa<number>` para números.